# Lab01 - Stanford Bunny Grid Benchmark

Repeated glDrawElements() calls - exploratory OpenGL performance experiment

**Repository:** ComputerGraphicsLabs
**Lab:** Lab01_BunnyGrid_BruteForce
**Baseline:** OpenGL 3.3 Core Profile, Visual Studio x64-Debug, VSync off

## 1. Purpose

This lab studies how FPS and frame time change as the number of Stanford Bunny copies increases. The same basic OpenGL workload was executed on a discrete NVIDIA GPU and an integrated AMD GPU.

This is an **exploratory rendering benchmark**, not a measurement of theoretical GPU performance. The implementation issues a separate draw call for every bunny, so CPU/driver submission overhead can become important at large object counts.

## 2. Environment and Tools

| Component | Configuration |
|---|---|
| API | OpenGL 3.3 Core Profile |
| Language | C++ |
| Libraries | GLFW + GLAD |
| IDE / build | Visual Studio, x64-Debug |
| Model | Stanford Bunny, indexed triangle mesh |
| Discrete GPU | NVIDIA GeForce RTX 4050 Laptop GPU |
| NVIDIA driver | 617.42 (reported by the program) |
| Integrated GPU | AMD Radeon(TM) Graphics |
| VSync | Off - glfwSwapInterval(0) |

## 3. Implementation

The program offers 1, 100, 1,000, or 10,000 bunnies. All objects reuse the same mesh. For each bunny, position/scale uniforms are updated and `glDrawElements()` is called. Objects are arranged in a grid.

```
for (int i = 0; i < bunnyCount; ++i) {
    glUniform2f(offsetLocation, x, y);
    glUniform1f(scaleLocation, scale);
    glDrawElements(GL_TRIANGLES, indexCount,
                   GL_UNSIGNED_INT, nullptr);
}
```

FPS is calculated from the number of frames rendered during a time window of approximately one second. Frame time is derived as 1000 / FPS in milliseconds.

## 4. Baseline Shading

The tested baseline used cyan/blue coloring with a simple gradient or fake-shine effect. Full Phong/Blinn-Phong lighting with stored normals and a specular term had not yet been introduced. The measurements therefore represent a baseline before more realistic lighting.

## 5. Measurements

| Bunnies | RTX 4050 FPS | RTX ms | AMD iGPU FPS | AMD ms |
|---|---|---|---|---|
| 1 | ~2,250 | ~0.44 | ~4,800 | ~0.21 |

| Bunnies | RTX 4050 FPS | RTX ms | AMD iGPU FPS | AMD ms |
|---|---|---|---|---|
| 100 | ~708 | ~1.41 | ~1,400 | ~0.71 |
| 1,000 | ~710 | ~1.41 | ~385 | ~2.60 |
| 10,000 | ~85 | ~11.7 | ~83 | ~12.0 |

Values are approximate and were taken from stabilized console readings. Short transient dips were not used as the primary values.

## 6. Observations

**1 bunny.** The AMD iGPU showed approximately twice the FPS. This does not imply that it is the more powerful GPU: the workload is very small and can be dominated by driver overhead, power states, hybrid-graphics routing, and CPU/driver interaction.

**100 bunnies.** The AMD iGPU remained approximately twice as fast in this particular baseline.

**1,000 bunnies.** The result reversed: RTX ~710 FPS versus AMD ~385 FPS. The larger workload begins to favor the discrete GPU.

**10,000 bunnies.** The results were nearly equal: RTX ~85 FPS and AMD ~83 FPS. This behavior is compatible with a substantial bottleneck from 10,000 uniform updates and draw calls per frame, so the benchmark is not measuring GPU shader throughput alone.

## 7. Clipping and Visibility Limitation

The current scene uses a fixed 10-column grid and fixed scale. At high object counts, some bunnies are partially clipped and many objects extend outside the visible region. OpenGL clips primitives against the clip volume after the vertex shader.

**Clipping is not culling.** The CPU still issues a draw call for every bunny even when an object is outside the visible region. Therefore, the 10,000-bunny result is best interpreted as a stress test of repeated draw submission rather than a benchmark in which all 10,000 bunnies are simultaneously visible.

## 8. Main Conclusion

Performance does not scale linearly with object count, and a stronger GPU is not necessarily faster in a very light or driver-limited workload. Overall performance depends on the CPU, OpenGL driver, draw-call count, uniform updates, rasterization workload, and GPU processing.

The near tie at 10,000 bunnies makes draw-call overhead an important **hypothesis to test**, not a proven explanation. A controlled follow-up experiment is required.

## 9. Next Experiments

| Experiment | Goal |
|---|---|
| GPU instancing | Compare 10,000 glDrawElements() calls with one glDrawElementsInstanced(..., 10000) call. |
| Phong / Blinn-Phong | Add normals, diffuse and specular lighting, then repeat measurements. |
| Frustum culling | Avoid submitting objects outside the view volume. |
| LOD | Use simpler meshes when an object is small or distant. |
| Combined pipeline | Evaluate instancing + culling + LOD. |
| Release build | Repeat the experiment using an optimized build instead of x64-Debug. |

## 10. Methodology Improvements

A stricter benchmark should use the same window size and resolution, VSync off, the same build mode, a warm-up period, and multiple repeated runs. Ideally, average FPS, median/percentile frame time, 1% lows, and CPU/GPU utilization should also be recorded.

For the Lab01 vs Lab02 comparison, both implementations should render an equivalent scene with the same mesh, object count, viewport, visibility conditions, and shader workload. This isolates the effect of replacing repeated draw calls with GPU instancing.

## Repository Progression

**Lab00:** basic Stanford Bunny rendering and project portability.
**Lab01:** repeated draw-call benchmark and performance baseline.
**Lab02:** GPU instancing and controlled comparison against Lab01.

This report documents the first benchmark as it was performed during development. Its limitations are preserved deliberately so that later labs can test and improve the rendering strategy.